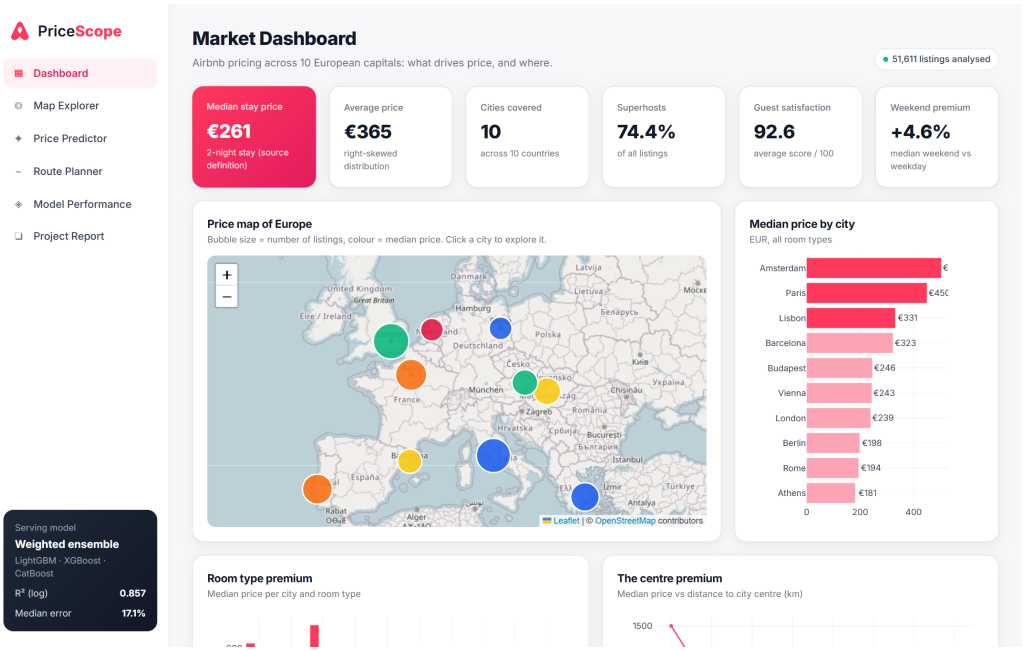
PROJECT REPORT

Airbnb Price Prediction Across 10 European Cities

Data cleaning, exploratory analysis, machine-learning modelling and deployment of **PriceScope**, an interactive price-intelligence web application



Dataset	51,707 listings, 10 cities, 29 columns
Best model	Weighted ensemble of CatBoost, XGBoost and LightGBM
Hold-out accuracy	R^2 0.857 (log price), median error 17.1%
Live application	https://airbnb-price-predictor-ivory.vercel.app
Source code	https://github.com/lovishv1718/airbnb-price-predictor
Date	30 October 2026

Table of Contents

1. Executive Summary 3

2. Introduction 3

3. Dataset 4

4. Data Cleaning and Validation 5

5. Exploratory Data Analysis 6

6. City Comparison 7

7. Feature Engineering and Methodology 8

8. Results 9

9. Error Analysis and Explainability 11

10. The PriceScope Web Application 13

11. Route Planner: Design and Analysis of Algorithms 17

12. Architecture, API and Deployment 22

13. Limitations and Future Work 22

14. Conclusion 23

1. Executive Summary

This project builds a complete, reproducible machine-learning system that estimates the price of an Airbnb listing in ten European cities from its location, its characteristics, its host and reviews, and the economic profile of its city. Starting from **51,707 listings** in Amsterdam, Athens, Barcelona, Berlin, Budapest, Lisbon, London, Paris, Rome and Vienna, the data was validated and cleaned to **51,611 rows**, analysed, and used to train and compare six models. The best model is served through PriceScope, a public web application with interactive maps.

Metric	Value	Meaning
R^2 (log price)	0.857	Share of variation in log price explained
Median absolute error	17.1%	Half of all predictions are closer than this
Within ±30%	74.4%	Share of predictions within 30% of the real price
MAE	€87	Mean absolute error (median price is €261)
Baseline (city median)	R^2 0.157	Quoting the city's median price

All scores come from listings the model never saw. The split is grouped by listing, so the weekday and weekend copies of the same apartment are never on both sides. A naive random split would report R^2 0.876, inflated by those duplicates.

- **Main finding:** location (centrality, attractions, restaurants, metro access) drives price most, followed by city, room type and capacity. Host reputation and reviews matter little.
- **Deliverables:** reproducible pipeline, six compared models, the PriceScope web app (dashboard, map explorer, price predictor, route planner, model page), public deployment on Vercel, and source code on GitHub.
- **Algorithms component:** a Route Planner that runs Dijkstra, Bellman-Ford and two Travelling Salesman algorithms (Held-Karp exact, nearest neighbour + 2-opt) over chosen listings, all written from scratch with tests and an empirical complexity analysis (Chapter 11).

2. Introduction

Most Airbnb hosts are individuals without revenue-management tools, and they set prices by looking at a few nearby listings and guessing. A data-driven estimate gives hosts a starting point, helps guests judge whether a price is fair, and helps analysts see how tourism demand is spread across a city. European capitals make a demanding test, because the model must learn both *between-city* differences (Amsterdam versus Athens) and *within-city* differences (the historic centre versus the suburbs).

2.1 Objectives

- Validate and clean the dataset, documenting every change.
- Explore how price varies with location, listing features and city economics.
- Train and compare regression models under a leakage-free evaluation protocol.
- Explain predictions globally and for each individual listing.
- Deliver the model in a professional, publicly deployed web application with maps.

2.2 Related work

The dataset comes from Gyódi and Nawaro (2021), who found that distance to the centre and the attraction and restaurant indices are among the strongest determinants of price. Hedonic-pricing studies (Rosen, 1974) model log price as a sum of characteristic effects, which explains why the log transform works well. Gradient-boosted trees (XGBoost, LightGBM, CatBoost) usually beat linear models on tabular pricing data because they capture non-linear effects and interactions, and SHAP values (Lundberg and Lee, 2017) are the standard way to explain them. A common pitfall in published Airbnb models is leakage through repeated listings. This project avoids it with grouped splits.

3. Dataset

The input file has 51,707 rows and 29 columns. Each row is one listing on a weekday or a weekend. The target, `price_total`, is the total price in euros of a 2-night stay for 2 people, as defined by the source dataset.

Group	Columns
Target	price_total
Listing	room_type, is_shared_room, is_private_room, max_guests, num_bedrooms
Host and reviews	is_superhost, is_multi_listing, is_business_listing, cleanliness_score (0-10), guest_satisfaction_score (0-100)
Location	latitude, longitude (rounded to 0.01°), distance_city_center, distance_metro, attraction_index_norm, restaurant_index_norm, proximity_index
Administrative	city, district, state, country_name
City economics	Crime_Index, Safety_Index, Monthly_Average_Net_salary, Taxi_price_per_Km, Monthly_Basic_Utilities
Time	day_type (weekday or weekend)

City	Country	Listings	Median price	Net salary	Safety index
London	United Kingdom	9,989	€239	€4,261	44.4
Rome	Italy	9,011	€194	€1,864	52.7
Paris	France	6,678	€450	€3,305	42.0
Lisbon	Portugal	5,753	€331	€1,305	67.4
Athens	Greece	5,263	€181	€1,164	44.8

City	Country	Listings	Median price	Net salary	Safety index
Budapest	Hungary	3,993	€246	€1,411	66.1
Vienna	Austria	3,530	€243	€3,135	71.6
Barcelona	Spain	2,833	€323	€2,103	48.1
Berlin	Germany	2,481	€198	€3,136	55.5
Amsterdam	Netherlands	2,080	€503	€4,502	74.3

4. Data Cleaning and Validation

The file had no missing values, but a systematic validation (`src/clean.py`, logged to `reports/cleaning_log.json`) still found several problems.

Step	Removed	Detail
Drop index column	0	'Unnamed: 0' was a pandas export index
Missing values check	0	0 missing cells found
Remove exact duplicates	63	-
Fix country label for Rome	0	4580 rows labelled 'Holy See (Vatican City State)' -> 'Italy'
Room-type flag consistency	0	room_type must agree with is_*_room flags
Domain range checks	0	-
Remove extreme price outliers	33	per-city 3xIQR fence on log(price); removes data-entry errors such as 43,955 EUR

- **Geocoding error:** 4,580 Rome listings were labelled *Holy See (Vatican City State)* and were relabelled *Italy*.
- **Outliers:** a per-city 3 × IQR fence on log price removed only 33 rows (0.06%), including a €43,955 entry error. Genuine luxury listings were kept.
- **proximity_index leakage check:** it is rebuilt from the four location columns with R^2 0.9994, and adding price as an input does not improve the fit. It is a valid feature.

Key finding: 18,774 listings appear twice, once on a weekday and once on a weekend, at almost the same price. Each row was given a `listing_id`, and every train/test split is grouped by it.

5. Exploratory Data Analysis

The cleaned data has a median price of €261 and a mean of €365. 74.4% of listings belong to Superhosts, and weekend stays carry a median premium of +4.6%.

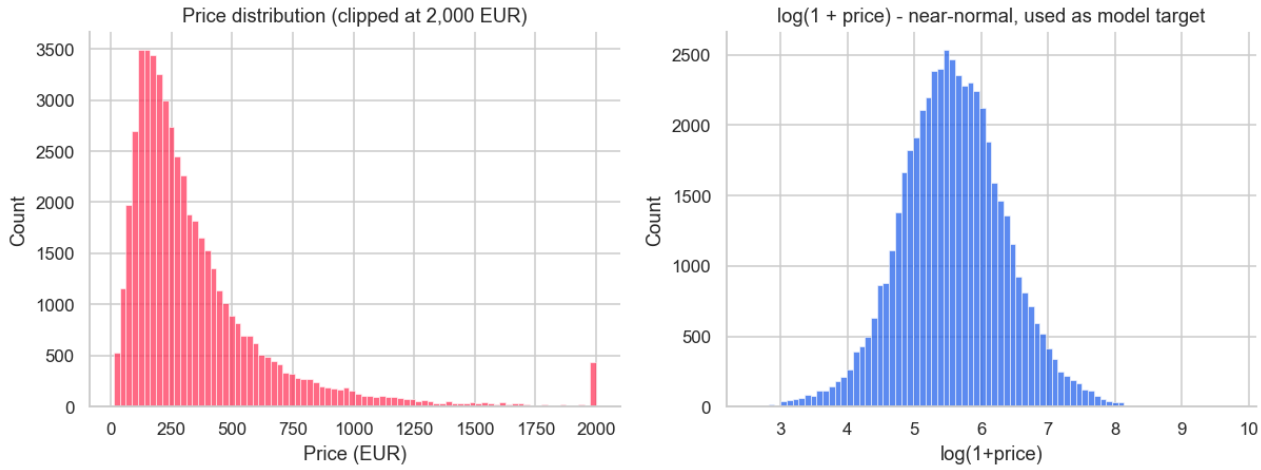


Figure 5.1 - Raw and log-transformed price.

Price is strongly right-skewed. On a log scale it is close to normal, so all models predict log(1 + price), and errors are measured in relative terms.

Figure 5.2 - Median price by city and room type.

Amsterdam (€503) and Paris (€450) are the most expensive markets, and Athens (€181) the cheapest. An entire home costs roughly 1.5 to 2.5 times as much as a private room in the same city, and the gap varies by city. That is an interaction tree models capture naturally.

Figure 5.3 - Log price against centre distance, attraction and restaurant indices.

Price falls steeply within the first 1 to 2 km of the centre, then flattens, and it rises with the attraction and restaurant indices. Location is the strongest signal in the data.

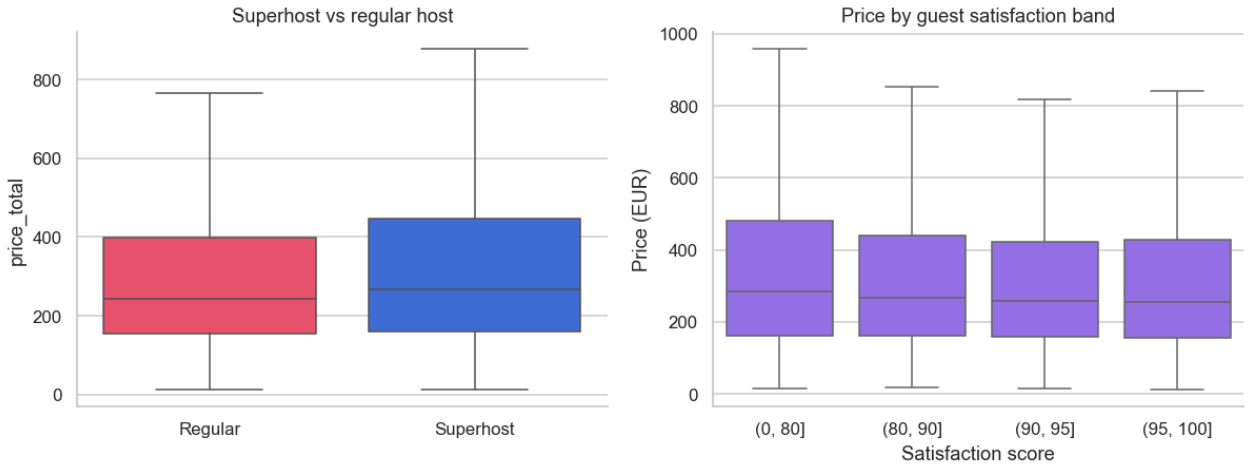


Figure 5.4 - Superhost versus regular host, and price by satisfaction band.

Superhost status and satisfaction scores come with only small price differences. Guests pay mainly for location and space; reviews probably affect occupancy more than price.

6. City Comparison

City	Listings	Median	Weekend	Entire home	≤1 km vs >4 km	Model R^2	Med. error
Amsterdam	2,080	€503	+9.3%	54.1%	6.1×	0.864	18.0%
Paris	6,678	€450	-1.2%	75.8%	3.0×	0.721	19.0%
Lisbon	5,753	€331	+0.2%	67.4%	4.5×	0.859	14.8%
Barcelona	2,833	€323	-2.7%	19.1%	3.7×	0.775	18.8%
Budapest	3,993	€246	+9.0%	89.6%	6.2×	0.745	19.3%
Vienna	3,530	€243	+5.1%	77.7%	4.8×	0.822	15.4%
London	9,989	€239	+3.8%	43.9%	9.3×	0.884	18.0%

City	Listings	Median	Weekend	Entire home	≤1 km vs >4 km	Model R^2	Med. error
Berlin	2,481	€198	+2.9%	35.5%	3.1×	0.857	17.8%
Rome	9,011	€194	+4.0%	61.7%	3.1×	0.796	15.6%
Athens	5,263	€181	-0.5%	92.3%	3.6×	0.777	17.3%

The centre premium (median price within 1 km of the centre divided by the median beyond 4 km) shows how concentrated each market is. Model accuracy is highest in Lisbon, Vienna and Rome. Paris has the lowest R^2, because prices there vary widely even at similar locations, probably due to building quality and views, which the data does not record.

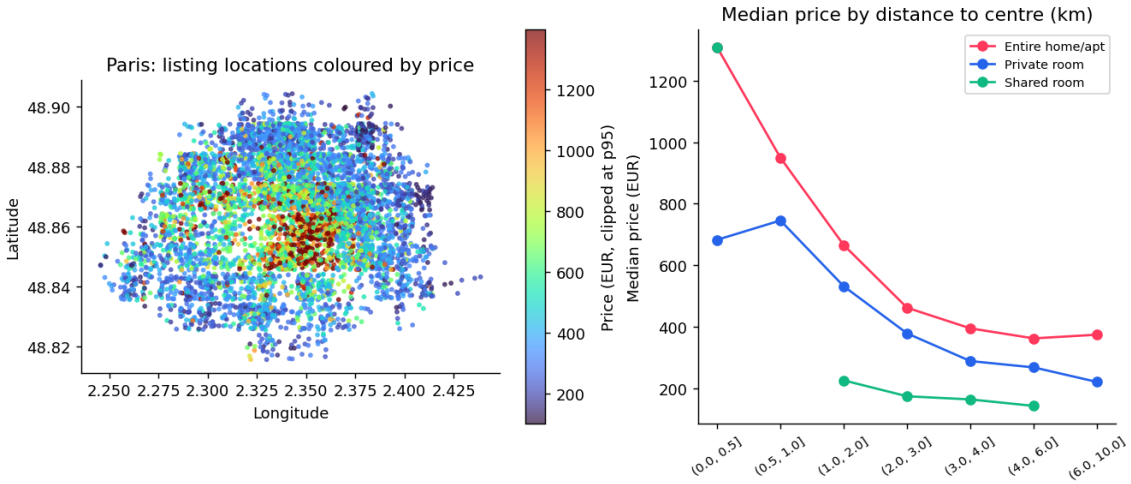


Figure 6.1 - Paris: listing prices on the map and the centre premium by room type.

7. Feature Engineering and Methodology

7.1 Features

The models use 25 features: the original listing, location and city columns plus six engineered ones. district, state and country_name were dropped as redundant with city and the coordinates.

Engineered feature	Formula	Rationale
is_weekend	day_type == weekend	Binary stay type
guests_per_bedroom	max_guests / max(bedrooms, 1)	How crowded the unit is
log_dist_center / metro	log(1 + distance)	Steep effect near the centre, flat further out
attr_x_rest	attraction × restaurant / 100	High only when a location has both
quality_score	0.4 × cleanliness × 10 + 0.6 × satisfaction	Combined review quality (0-100)

Spatial neighbour price features (out-of-fold k-nearest-neighbour target encoding) were also tested. They did not improve hold-out accuracy and were removed.

7.2 Evaluation protocol

- **Target:** log(1 + price), converted back to euros for the euro metrics.
- **Hold-out test set:** 10,366 rows (20% of listings), grouped by listing_id and never used for tuning.
- **Validation:** a further grouped 15% of training listings, used for early stopping and ensemble weights.
- **Production model:** refit on all 51,611 rows with the tuned number of boosting rounds.

7.3 Models

Model	Configuration
City median	Baseline: median training price of the listing's city
Ridge regression	One-hot encoding, standardised features, alpha 1.0
Random Forest	300 trees, max_features 0.5, min_samples_leaf 2
LightGBM	63 leaves, lr 0.02, subsampling, 1032 rounds (early stopping)
XGBoost	depth 8, lr 0.02, native categoricals, 656 rounds
CatBoost	depth 8, lr 0.04, native categoricals, 1542 rounds
Weighted ensemble	Non-negative weights minimising validation MSE: LightGBM 17%, XGBoost 42%, CatBoost 42%

8. Results

Model	R^2 log	R^2 €	MAE €	MAPE	Med. APE	±20%	±30%
City median (baseline)	0.1572	0.040	191.4	66.7%	42.2%	25.3%	37.2%
Ridge regression	0.8108	0.615	99.6	25.6%	20.7%	48.5%	67.0%
Random Forest	0.8469	0.635	90.1	22.3%	17.2%	56.0%	73.1%
LightGBM	0.8542	0.651	88.1	22.0%	17.5%	55.9%	74.1%
XGBoost	0.8553	0.650	87.6	21.8%	17.1%	56.6%	74.2%
CatBoost	0.8558	0.653	87.3	21.8%	17.3%	56.4%	74.2%
Weighted ensemble	0.8569	0.653	87.1	21.7%	17.1%	56.5%	74.4%

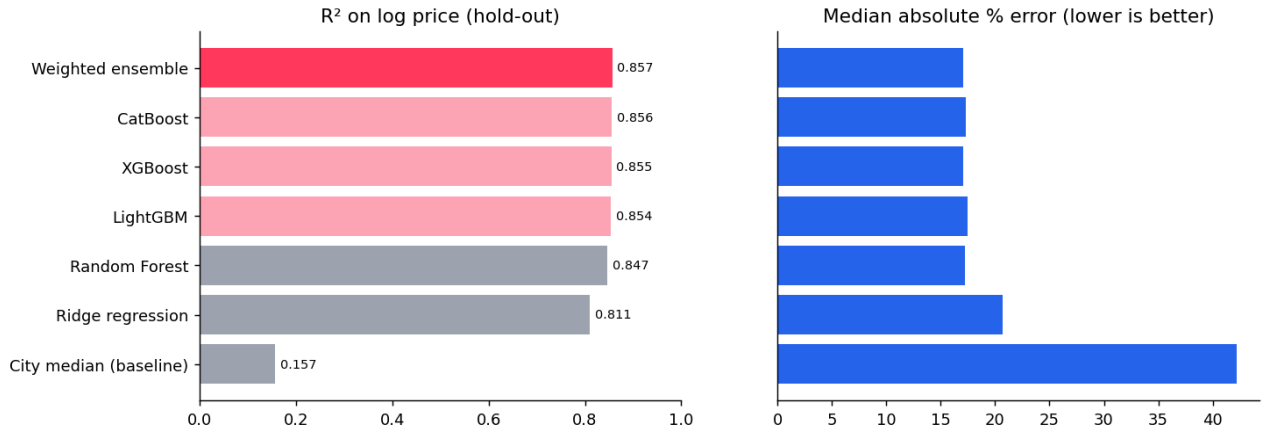


Figure 8.1 - R^2 and median error for every model on the hold-out set.

Every learned model is far better than the city-median baseline. Ridge already reaches R^2 0.81 because many effects are close to linear on a log scale. The tree ensembles add about 4-5 points by capturing non-linear distance effects and interactions. The three boosting libraries score almost identically, which suggests the model is close to the limit of what these features can explain, and the ensemble is slightly better on every metric.

Why is R^2 in euros lower (0.65)? Squared euro error is dominated by a handful of very expensive listings. On the log scale, which reflects relative accuracy, the model explains 86% of the variation. The 17% median error is the best summary of a typical prediction.

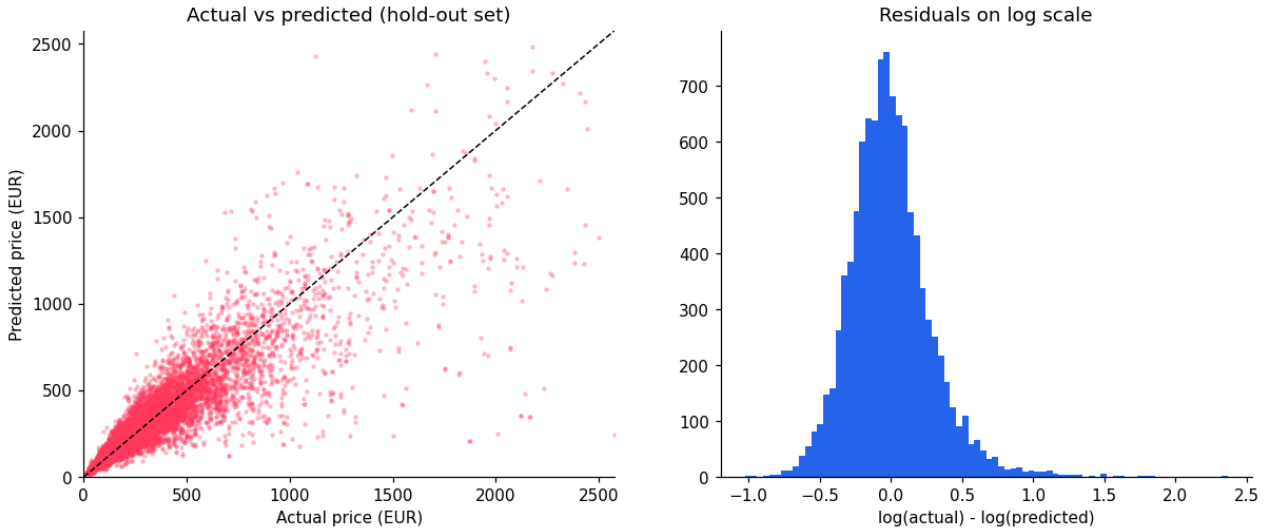


Figure 8.2 - Predicted against actual price, and the log residuals.

9. Error Analysis and Explainability

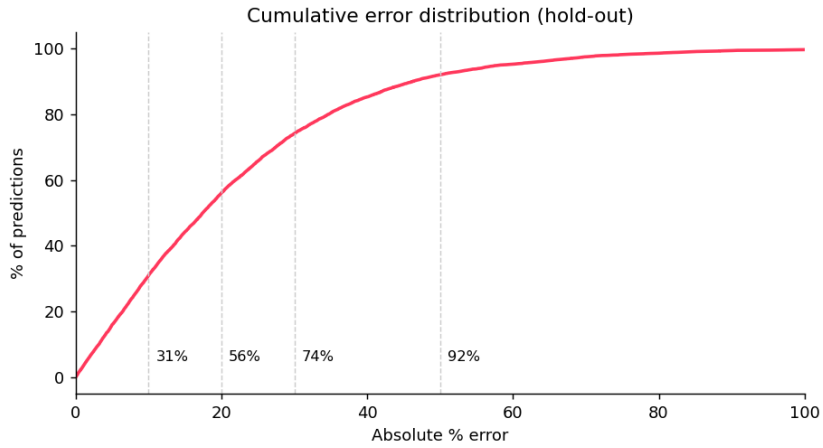


Figure 9.1 - Cumulative share of predictions by absolute % error (LightGBM, hold-out).

30.9% of predictions are within ±10% of the real price, 56.1% within ±20% and 74.1% within ±30%. The model is most accurate in the middle of the price range. It slightly over-predicts the cheapest listings and under-predicts the most expensive ones, because the features that would justify an extreme price (interior, view, size) are not in the data. Entire homes are predicted best (16.8% median error), then private rooms (18.7%).

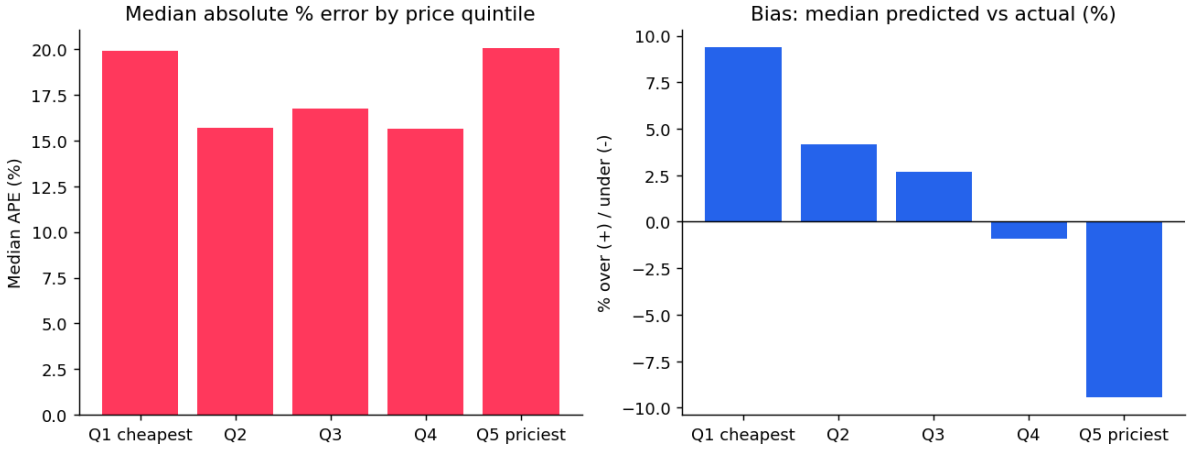


Figure 9.2 - Error and bias by price quintile.

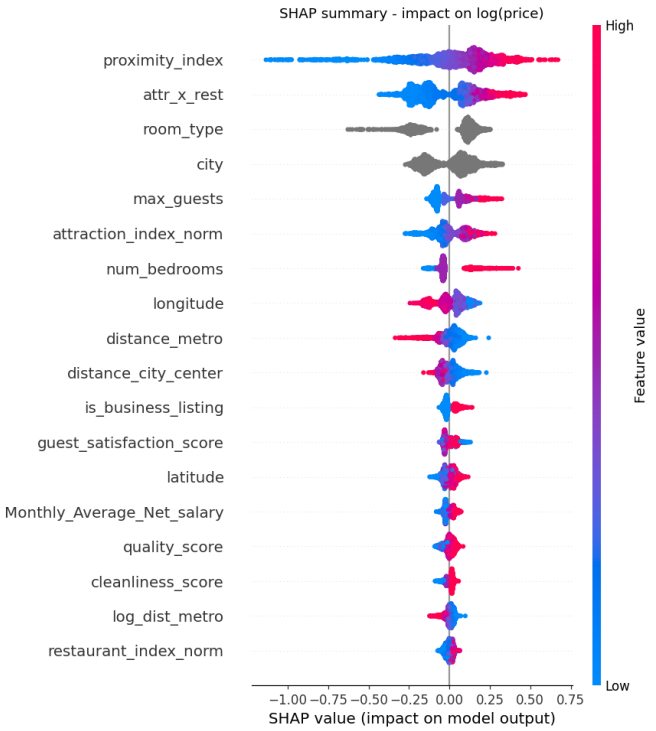


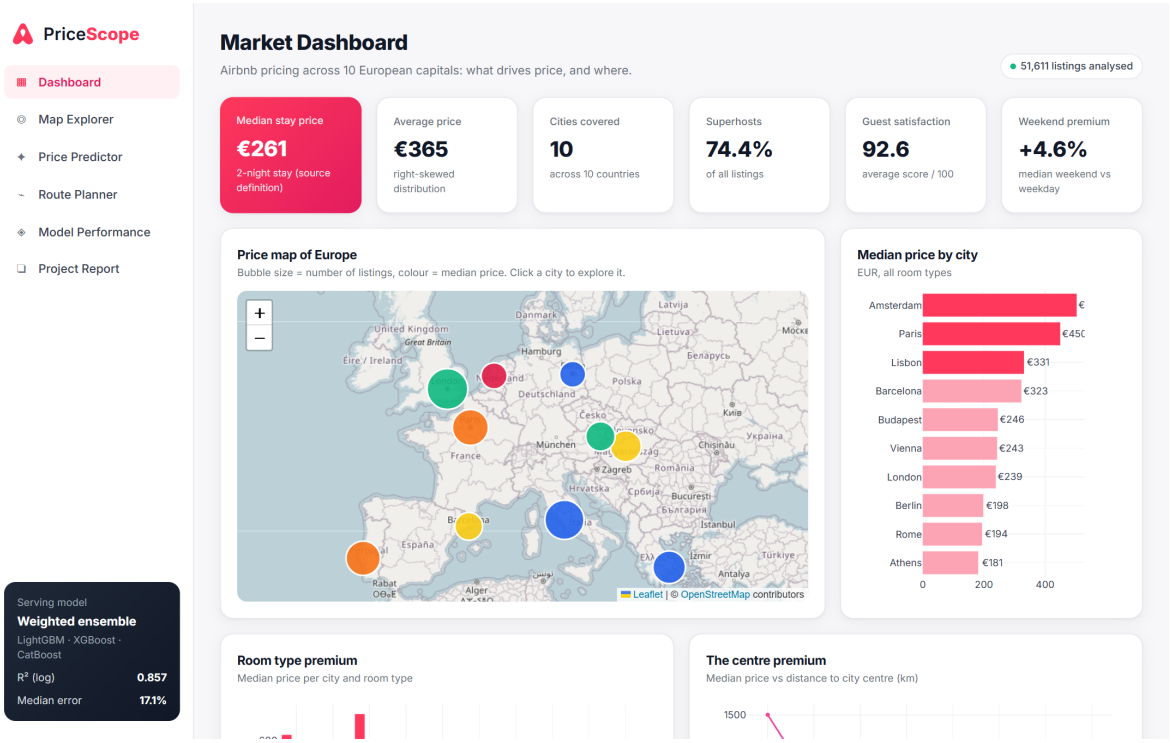
Figure 9.3 - SHAP summary: impact of each feature on log price.

The leading features, proximity_index and attr_x_rest, together account for 46% of LightGBM's split gain, and both measure how central and lively a location is. High values push the price up strongly, as do entire homes and more guests and bedrooms. Long distances to the centre and the metro lower it. In the app, these contributions are grouped into nine readable price drivers for every prediction.

10. The PriceScope Web Application

PriceScope is a Flask application with a responsive interface: a sidebar for navigation, card layouts, Plotly charts and Leaflet maps with OpenStreetMap tiles. It has six pages and a REST API. The Route Planner page is covered in Chapter 11.

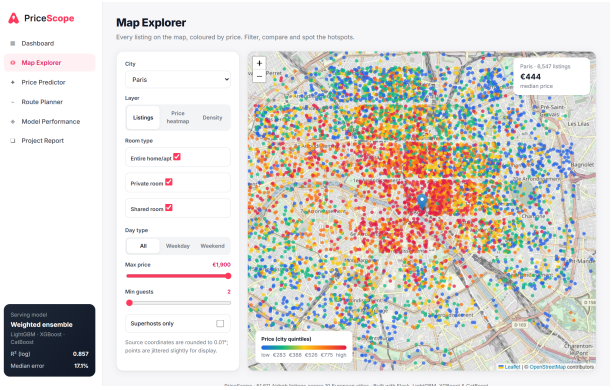
10.1 Market dashboard



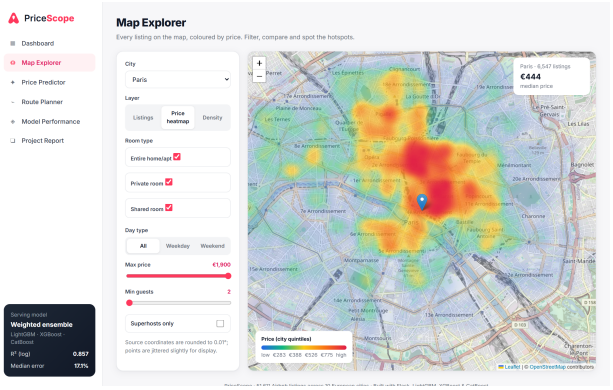
Screenshot 10.1 - Dashboard: KPI cards, Europe price map and median price by city.

Six KPI cards summarise the market. A map of Europe shows one bubble per city, sized by listings and coloured by median price, and clicking a bubble opens that city in the explorer. Further down are charts of room-type premiums, the centre premium, the price distribution and capacity, and a city scorecard.

10.2 Map explorer



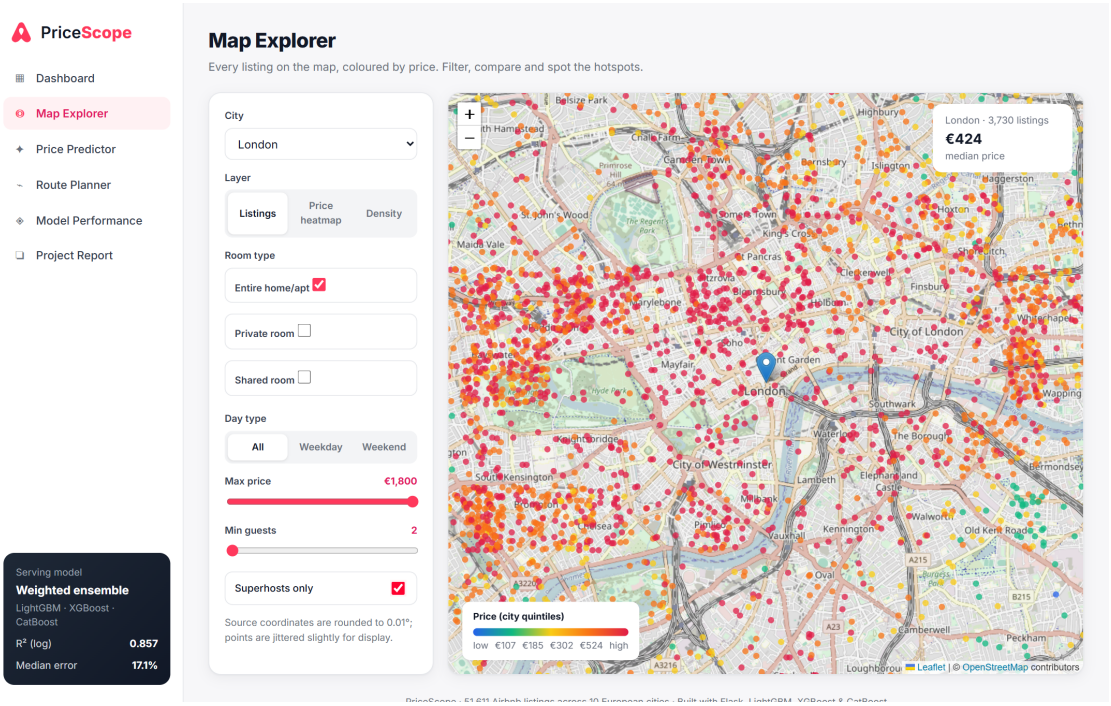
Listings layer



Price heatmap layer

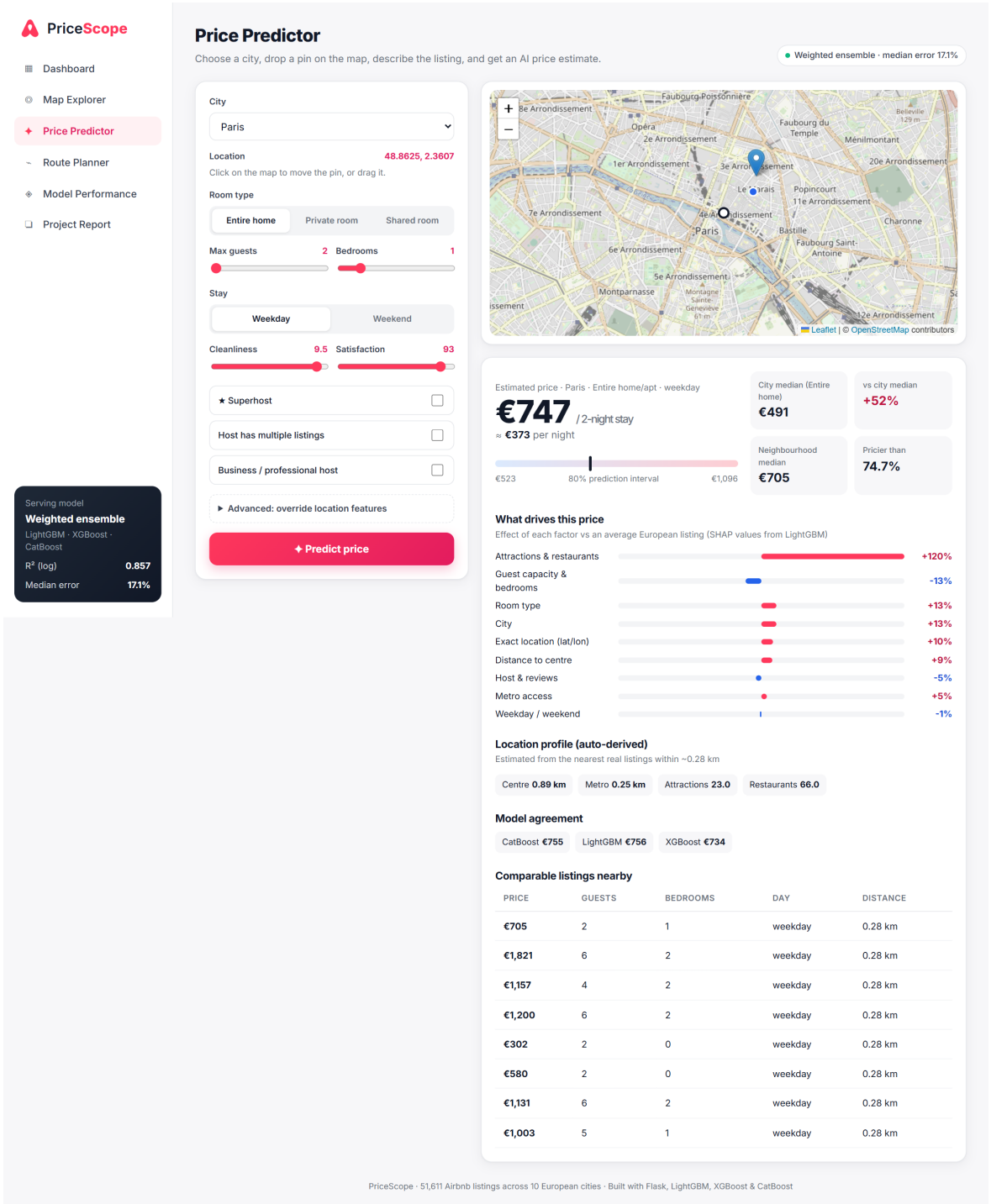
Screenshot 10.2 - Map Explorer, Paris.

Every listing in the chosen city is drawn on the map, coloured from blue (cheapest 20%) to red (most expensive 20%). The price heatmap highlights expensive districts, and a density layer shows where supply is concentrated. Filters for room type, day type, maximum price, guests and Superhost status update the map and the summary card instantly.



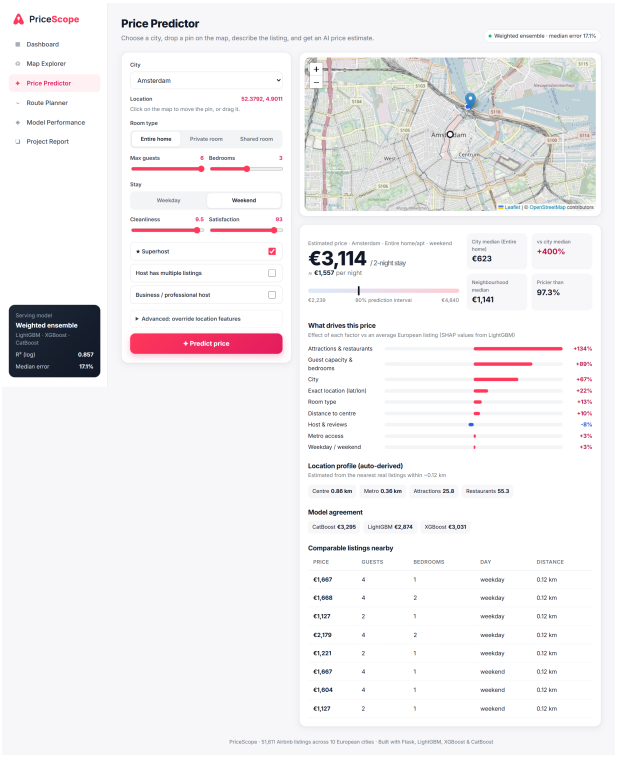
Screenshot 10.3 - London filtered to entire homes owned by Superhosts.

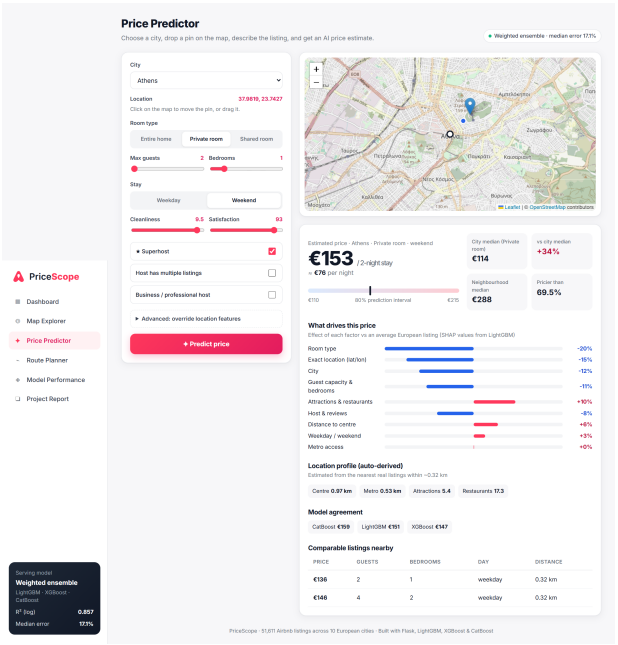
10.3 Price predictor



Screenshot 10.4 - Full prediction for a central Paris apartment.

The user picks a city, clicks or drags the pin to the listing's location and describes it. Metro distance and the attraction, restaurant and proximity indices are derived from the 25 nearest real listings. Distance to the centre is computed from a city centre estimated from the data. The result shows the 2-night and per-night price, an 80% prediction interval, comparisons with the city and neighbourhood medians, the price drivers, each model's prediction and nearby comparable listings.

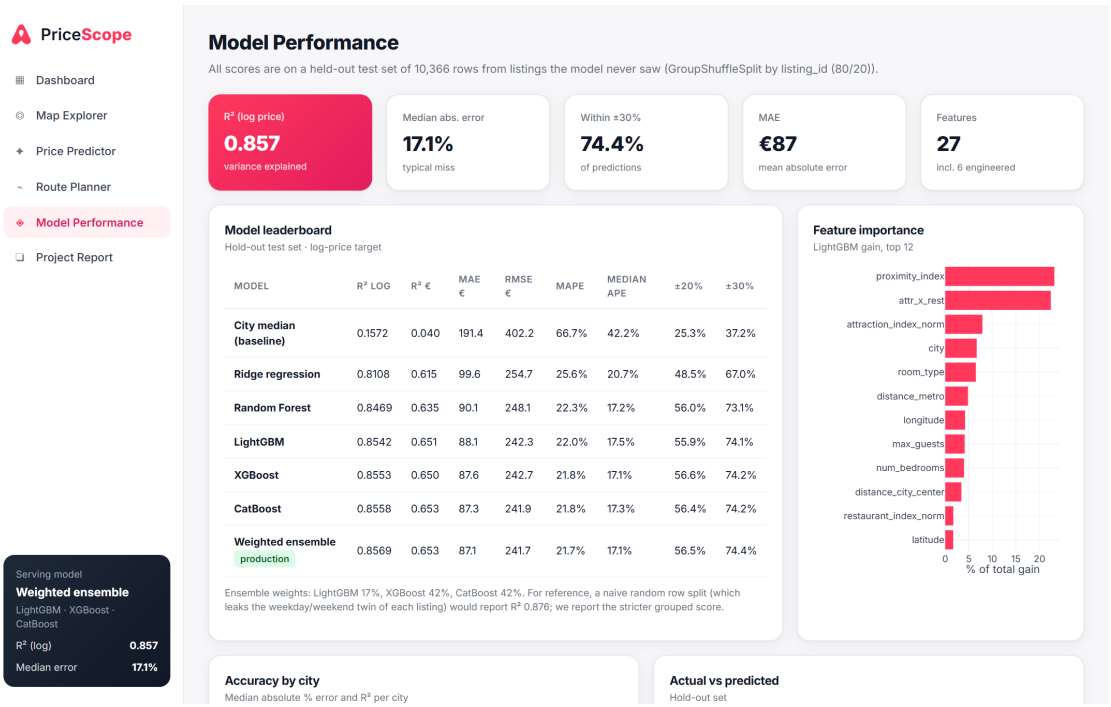




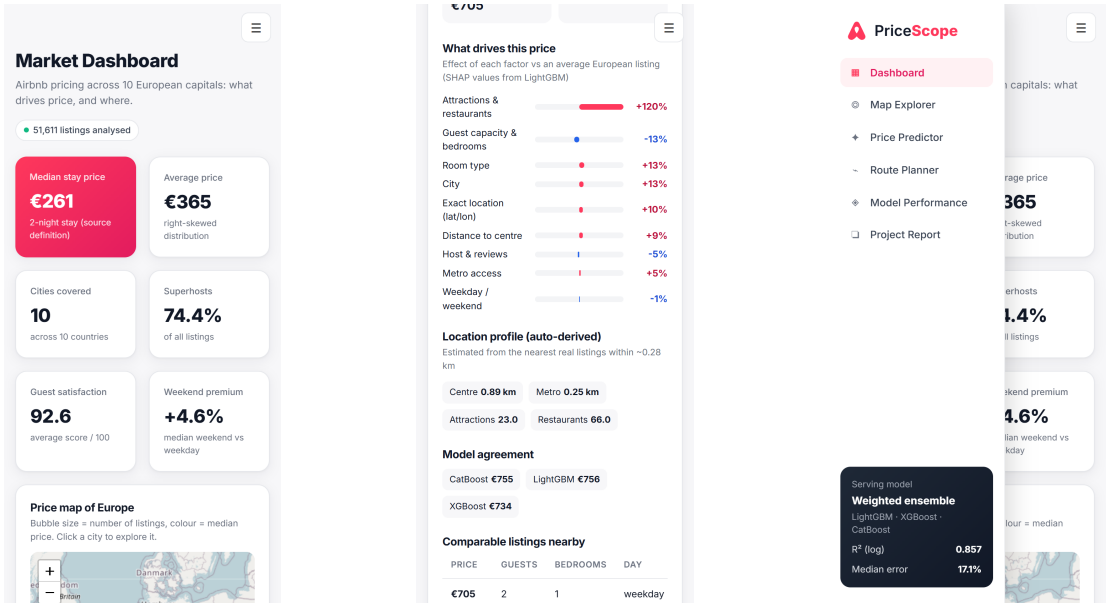
Amsterdam: 6-guest family home, weekend *Athens: private room for 2*

Screenshot 10.5 - Two contrasting prediction scenarios.

10.4 Model performance and mobile views



Screenshot 10.6 - Model Performance page: KPIs, leaderboard and feature importance.



Dashboard *Prediction result* *Navigation menu*

Screenshot 10.7 - Mobile views (390 × 844).

11. Route Planner: Design and Analysis of Algorithms

The Route Planner turns the listings into a graph problem. A traveller picks the Airbnbs they want to visit; the app finds the shortest path between two of them and the shortest round trip that visits all of them. Every algorithm is implemented from scratch in plain Python in `src/algorithms.py`, using only heapq from the standard library, and served through `POST /api/route`.

11.1 Problem formulation

- **Vertices V:** the n selected listings (2 to 20), one of them marked as the start.
- **Edges E:** each listing is joined to its k nearest listings (k = 1 to 6, default 3). The edge weight is the great-circle (haversine) distance in km. The graph is undirected, so $|E| \leq k \cdot n$.
- **Connectivity:** a k-nearest-neighbour graph can split into clusters. Depth-first search finds the components, and the closest pair of listings between components is joined until one remains.
- **Shortest path:** from the start to every listing, solved by Dijkstra and by Bellman-Ford.
- **Travelling Salesman (TSP):** a minimum-length closed tour from the start through every listing. It runs on the all-pairs shortest-path matrix, so each leg of the tour follows a real path in the graph.

11.2 Algorithms and complexity

Algorithm	Technique	Time	Space
k-NN graph + DFS	sort neighbours, then join components	$O(V^2 \log V)$	$O(V + E)$
Dijkstra	greedy, binary min-heap, lazy deletion	$O((V + E) \log V)$	$O(V + E)$
Bellman-Ford	dynamic programming over edge relaxations	$O(V \cdot E)$	$O(V)$
All-pairs Dijkstra	Dijkstra from every vertex	$O(V (V + E) \log V)$	$O(V^2)$
Held-Karp TSP	dynamic programming over subsets (bitmasks)	$O(n^2 \cdot 2^n)$	$O(n \cdot 2^n)$
Nearest neighbour + 2-opt	greedy construction + local search	$O(n^2)$ per pass	$O(n)$
Brute-force TSP (tests only)	try every permutation	$O(n!)$	$O(n)$

Dijkstra(G, s)

```
dist[v] = INF for all v;  dist[s] = 0;  heap = [(0, s)]
while heap not empty:
    (d, u) = pop_min(heap)
    if u settled: continue          # stale entry
    settle u
    for (v, w) in adj[u]:
        if d + w < dist[v]:
            dist[v] = d + w;  prev[v] = u;  push(heap, (dist[v], v))
```

Bellman-Ford(G, s)

```
dist[v] = INF for all v;  dist[s] = 0
repeat V-1 times:
    changed = false
    for each directed edge (u, v, w):
        if dist[u] + w < dist[v]: dist[v] = dist[u] + w;  prev[v] = u;  changed = true
    if not changed: break           # early exit
negative_cycle = any(dist[u] + w < dist[v] for (u, v, w))
```

Held-Karp(D, start)

```
C[{j}][j] = D[start][j]
C[S + {k}][k] = min over j in S of  C[S][j] + D[j][k]     (k not in S, S + {k} = union)
tour length = min over j of  C[all][j] + D[j][start]
the tour is rebuilt from the stored parent of each state
```

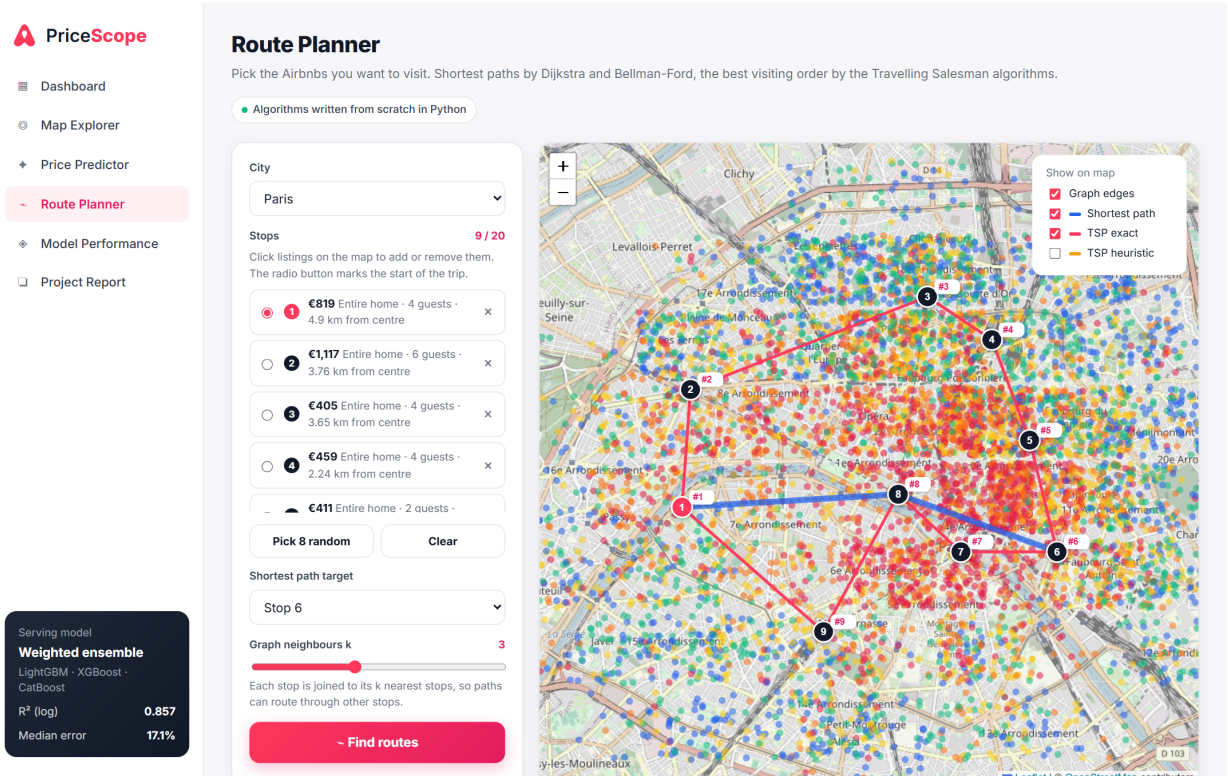
2-opt(tour, D)

```
repeat until no improvement:
    for every pair of edges (a,b), (c,d) in the tour:
        if D[a][c] + D[b][d] < D[a][b] + D[c][d]:
            reverse the segment b..c       # removes a crossing
```

Dijkstra is correct only for non-negative weights, which holds for distances. Bellman-Ford is slower but also handles negative weights and reports negative cycles; it is included to compare the two approaches and as an independent check, since both must return identical distances. The TSP is NP-hard: Held-Karp gives the guaranteed optimum but needs exponential time, so the app uses it up to 12 stops. Nearest neighbour + 2-opt is polynomial and very fast, but it can get stuck in a local optimum.

11.3 Worked example: nine Paris landmarks

The example uses the Paris listing closest to each of nine landmarks, starts at the Eiffel Tower and finds the shortest path to Bastille. With k = 3, the graph has 9 vertices and 18 edges.



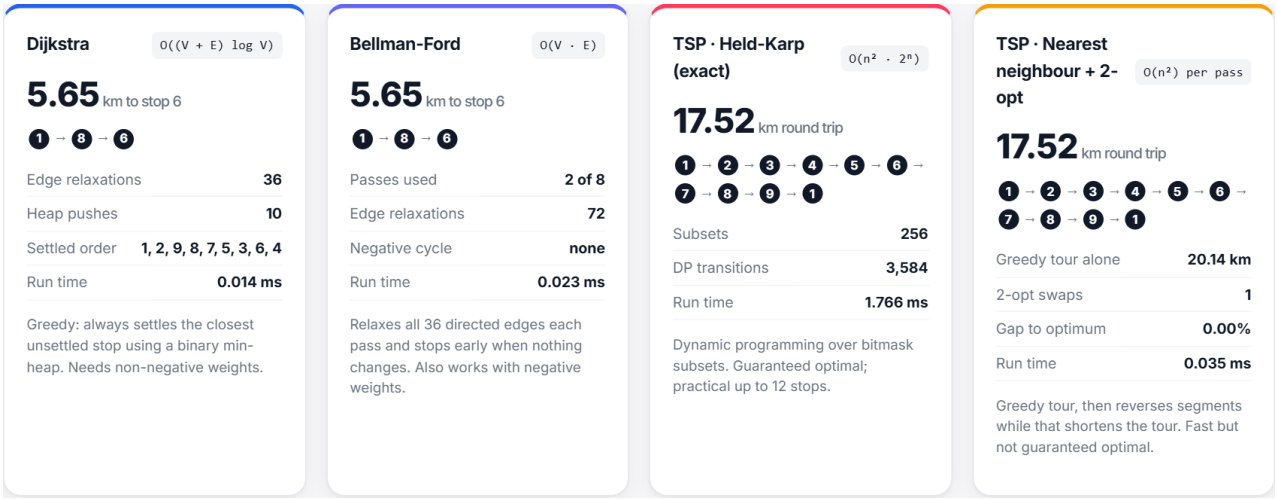
Screenshot 11.1 - Route Planner: exact TSP tour (red), Dijkstra path to Bastille (blue) and numbered stops.

Stop	Near	Price	Dijkstra km	Bellman-Ford km	Shortest path
1	Eiffel Tower	€819	0.000	0.000	1
2	Arc de Triomphe	€1,117	1.725	1.725	1 → 2
3	Sacré-Cœur	€405	5.446	5.446	1 → 2 → 3
4	Gare du Nord	€459	5.827	5.827	1 → 8 → 4
5	République	€411	5.251	5.251	1 → 8 → 5
6	Bastille	€710	5.649	5.649	1 → 8 → 6
7	Notre-Dame	€1,097	4.417	4.417	1 → 8 → 7
8	Louvre	€525	3.180	3.180	1 → 8
9	Montparnasse	€422	2.765	2.765	1 → 9

Algorithm	Result	Route	Work done
Dijkstra	5.65 km to stop 6	1 → 8 → 6	36 relaxations, 10 heap pushes
Bellman-Ford	5.65 km to stop 6	1 → 8 → 6	2 of 8 passes, 72 relaxations, no negative cycle
Held-Karp	17.52 km round trip	1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 1	256 subsets, 3,584 DP transitions

Algorithm	Result	Route	Work done
NN + 2-opt	17.52 km round trip	1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 1	greedy 20.14 km, then 1 2-opt swap(s)

Both shortest-path algorithms return identical distances to every stop. The shortest route from the Eiffel Tower to Bastille goes through the Louvre listing (5.65 km), because the two have no direct edge. Bellman-Ford needed only 2 of its 8 allowed passes. The optimal round trip is 17.52 km and follows the landmarks in a ring. The greedy tour alone was 20.14 km (15% longer); one 2-opt swap removed its crossing and reached the optimum.



Screenshot 11.2 - Result cards for the four algorithms, with complexity, operation counts and run times.

11.4 Empirical analysis

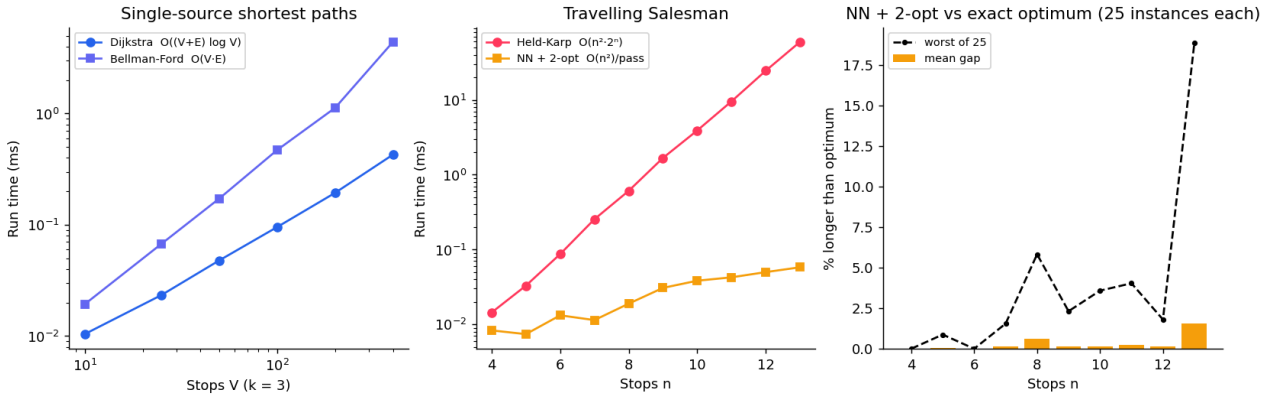


Figure 11.1 - Measured run times (log scale) and the 2-opt optimality gap on random Paris-sized instances.

Stops V	Edges E	Dijkstra ms	Bellman-Ford ms	Ratio
10	22	0.010	0.019	1.9×
25	47	0.023	0.067	2.9×
50	92	0.048	0.172	3.6×
100	185	0.095	0.470	4.9×
200	378	0.194	1.124	5.8×
400	751	0.427	4.409	10.3×

Shortest paths. Dijkstra grows roughly linearly here, because E ≈ 2V in a k = 3 graph. Bellman-Ford is 10× slower at 400 stops. Thanks to the early exit it needs far fewer than V − 1 passes on these geometric graphs, so it stays well below its O(V·E) worst case, but it still re-scans every edge on each pass.

Stops n	4	5	6	7	8	9	10	11	12	13
Held-Karp ms	0.01	0.03	0.09	0.25	0.61	1.67	3.90	9.55	24.51	59.55
NN + 2-opt ms	0.01	0.01	0.01	0.01	0.02	0.03	0.04	0.04	0.05	0.06
Mean gap %	0.00	0.03	0.00	0.15	0.61	0.12	0.15	0.26	0.12	1.57
Optimal in %	100	96	100	88	68	92	88	88	88	80

Travelling Salesman. Held-Karp's time multiplies by about 2.5× for each extra stop, close to the factor of about 2 predicted by $O(n^2 \cdot 2^n)$, and reaches 60 ms at 13 stops. At that rate, 20 stops would take minutes, which is why the app switches to the heuristic above 12. Nearest neighbour + 2-opt stays below 0.1 ms. Across 25 random instances per size, it found the exact optimum 68-100% of the time, with a mean gap below 2%. Its worst case was 19% longer than optimal, which shows why an exact method is still worth having when it is affordable.

11.5 Correctness testing

`tests/test_algorithms.py` has 11 unit tests, all passing (`python -m unittest discover tests`):

- Dijkstra and Bellman-Ford match an independent Floyd-Warshall implementation on 30 random graphs, from every source.
- A hand-worked 4-node example where the shortest path (A-C-B-D, length 4) is not the direct edge.
- Bellman-Ford handles negative edges correctly and detects a negative cycle.
- Held-Karp equals the O(n!) brute-force optimum for 3 to 8 stops.
- 2-opt never beats the exact optimum, and every tour visits each stop exactly once.
- Expanded tours use only real graph edges; the generated graphs are connected and symmetric.
- The API rejects fewer than 2 or more than 20 stops, out-of-range indices and malformed input.

12. Architecture, API and Deployment

```
Cleaned_data.csv
  -> src/clean.py        dedupe, fix labels, checks, outlier fence, listing_id
  -> src/eda.py          figures + dashboard data
  -> src/train.py        grouped split, 5 models + ensemble, metrics, SHAP
  -> src/export_lite.py  LightGBM -> numpy arrays, city centres
  -> app.py + src/predictor.py   PriceScope (local: ensemble | Vercel: lite LightGBM)
```

Endpoint	Description
GET /api/health	Service status, serving model and its R^2
GET /api/listings/<city>	All listings of a city in compact form (used by the map)
POST /api/predict	Price, interval, drivers, location profile and comparables. Needs city, latitude and longitude; other fields have defaults
POST /api/route	Graph, Dijkstra, Bellman-Ford, Held-Karp and 2-opt results for 2-20 points (Chapter 11)

Deployment. The source code is on GitHub (lovishv1718/airbnb-price-predictor) and the app runs on Vercel at **https://airbnb-price-predictor-ivory.vercel.app**. XGBoost, CatBoost and SHAP together exceed Vercel's bundle limit of about 250 MB, so the public site serves the trained LightGBM model exported to numpy arrays (R^2 0.854 against 0.857 for the ensemble). Its predictions matched LightGBM exactly on 300 test listings, at about 10 ms each. Locally, the app serves the full ensemble.

Test	Result
No listing in both train and test	Passed (0 overlap)
proximity_index leakage	None (R^2 0.9994 with or without price)
Lite model against LightGBM	Maximum difference 0.0
All pages and API, locally and on Vercel	HTTP 200, no console errors
Map, filter and prediction interactions (Playwright)	Working (screenshots in Chapter 10)
Mobile layout (390 px)	Stacks correctly, menu works
Route Planner algorithms (11 unit tests)	All pass (Section 11.5)

13. Limitations and Future Work

- **Missing price drivers:** photos, floor area, amenities, interior and views are not in the data and probably explain most of the remaining 14% of variance.
- **Coarse coordinates:** latitude and longitude are rounded to about 1 km, so fine location detail comes from the distance and index columns.
- **Snapshot data:** there is no seasonality or booking calendar, so prices should be re-validated before live use.
- **Straight-line routes:** the Route Planner uses great-circle distances between listings, not street-network distances. Real walking or driving distances would need a road graph such as OpenStreetMap's.

- **Future work:** street-network routing for the Route Planner, text and photo features, quantile models for listing-specific intervals, Optuna tuning with grouped cross-validation, data from several snapshots over time, an occupancy model for revenue-optimal pricing, and ONNX export of the full ensemble.

14. Conclusion

The project delivered an end-to-end system for Airbnb price prediction across ten European cities. Validating the supposedly clean data found duplicates, a geocoding error affecting 4,580 Rome listings, extreme price errors and 18,774 listings duplicated across weekday and weekend rows. That last finding shaped an evaluation protocol that measures performance on genuinely new listings.

Location turned out to be the decisive factor in price. A weighted ensemble of CatBoost, XGBoost and LightGBM explains 85.7% of the variation in log price on unseen listings, with a typical error of 17.1%. PriceScope makes this usable: drop a pin, describe the listing, and get a price with an uncertainty range, an explanation and real comparable listings. The Route Planner adds a Design and Analysis of Algorithms component: Dijkstra, Bellman-Ford, Held-Karp and 2-opt, implemented from scratch, verified by unit tests and compared empirically against their theoretical complexity. The system is publicly deployed and fully reproducible from the GitHub repository.

References

- Gyódi, K. and Nawaro, Ł. (2021). Determinants of Airbnb prices in European cities: A spatial econometrics approach. *Tourism Management*, 86, 104319.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. *NeurIPS* 30.
- Ke, G. et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *NeurIPS* 30.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD '16*, 785-794.
- Prokhorenkova, L. et al. (2018). CatBoost: unbiased boosting with categorical features. *NeurIPS* 31.
- Rosen, S. (1974). Hedonic prices and implicit markets. *Journal of Political Economy*, 82(1), 34-55.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1, 269-271.
- Bellman, R. (1958). On a routing problem. *Quarterly of Applied Mathematics*, 16(1), 87-90.
- Held, M. and Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the SIAM*, 10(1), 196-210.
- Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6), 791-812.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C. (2022). *Introduction to Algorithms*, 4th ed. MIT Press.